

Inventory Management System
A Project Synopsis Report Submitted To



SAGE UNIVERSITY, INDORE
Towards Partial Fulfillment for the Award of
Bachelor of Technology
in
Computer Science & Engineering

SUBMITTED BY:

NAMAMI TIWARI
[23ADV3CST0080]

SUBMITTED TO :

Dr. Avinash Raipuriya

Institute of Advance Computing core
SAGE University
Bypass Road, Kailod Kartal, Indore
Madhya Pradesh - 452020
June-July 2026

CONTENTS

1. The Aim Of The Project
 - 1.1. Title of Project
 - 1.2. Introduction
2. The Need of The Project
 - 2.1. Problem Definition
 - 2.2. Problem Solution
3. Objectives and scope of Project
4. Methodology
5. Process Description
6. Hardware and Software Requirement
7. Resources and Limitations
8. Future Work
9. Project Schedule
10. Conclusions.
11. References.

1. The Aim of The Project

1.1. Title of the Project

Inventory Management System

1.2. Introduction

The Inventory Management System is a web-based platform that connects business operations with real-time tracking of product stock levels, procurement, and sales. Users can view real-time stock availability, record stock-in and stock-out transactions, and receive automated low-stock alerts based on current inventory levels. The system provides functional features such as product catalog management, automated reorder notifications, dashboard tracking, and comprehensive sales reports to ensure transparency and efficiency. The platform aims to simplify the inventory tracking process, reduce manual errors for businesses, and provide a reliable environment for online supply chain and stock facility management.

2. The Need of the Project

2.1. Problem Definition

The retail and supply chain industry has grown rapidly, but many traditional inventory setups face challenges such as manual tracking errors, lack of real-time visibility, communication gaps, and stock reconciliation disputes. Businesses often struggle to track accurate product counts, leading to wasted time and lost sales, while facility owners find it difficult to track and manage their stock levels efficiently. There is a clear need for a centralized platform that provides secure product management, live availability tracking, and transparent transaction logging. The proposed Inventory Management System aims to solve these issues by integrating real-time stock dashboards, automated low-stock alerts, reporting features, and interactive user management interfaces.

2.2. Problem Solution

The Inventory Management System solves these issues by creating a centralized web application. The software eliminates the need for manual paperwork and guesswork by offering real-time visibility into current stock levels. Business users can securely track, update, and record specific product transactions before running out of stock, while facility operators receive a dedicated control panel to monitor their catalog capacities and manage transaction histories smoothly.

3. Objective and Scope of the Project

Objectives

The main goals of the system are:

- To create a simple platform that connects business operations with real-time stock levels.
- To let warehouse managers show their available products, categories, and SKUs.

- To help users search for product items and record a transaction before stock runs out.
- To provide low-stock alerts for holding minimum required inventory securely.
- To include a safe backend system for easy digital transaction recording.
- To use clear dashboards so users can easily find the exact stock levels of the product.
- To send real-time alerts and messages about low-stock and automated reorder statuses.
- To allow users to generate comprehensive reports and summaries based on sales data.

Scope

The project can be used by:

- Everyday retail store staff and cashiers looking for quick product lookups.
- Private warehouse and store owners who want to manage their stock online.
- Shopping malls, retail outlets, and distribution centres.
- Office supply rooms and small business units handling commercial inventory.

4. Methodology

The system is developed using a modern web application stack, splitting the project into a user interface (Frontend) and a server-side system (Backend and Database) to ensure fast and reliable performance.

4.1. Technology Stack

- **User Interface (Frontend):** Built using **HTML5**, **CSS3**, and **JavaScript** to create a responsive, data-focused business interface and clean dashboard for tracking products.
- **Server Logic (Backend):** Powered by **Node.js** and **Express.js** to handle product updates, automated low-stock calculations, and smooth communication between the frontend and the database.
- **Database Management:** Managed via **MongoDB** or **MySQL** to safely store product catalogs, supplier details, livestock log states, and transaction records.

4.2. Main Features & Workflow

- **Authentication Module:** Secure registration and login portals with separate roles for staff members, inventory managers, and system administrators.
- **Product Catalog Engine:** A structured search and filter tool that lets users input item names or SKUs to find specific categories, pricing, and exact stock locations.
- **Live Stock Matrix:** An interactive visual layout table showing exactly which products are green (sufficient stock), red (out of stock), or yellow (low stock alert level).
- **Transaction & Notification Gateway:** An automated stock recording sequence where a user logs an entry or exit transaction, real-time quantities update, and automated reorder alerts generate confirmation summaries.

5. Process Description

The operation of the application is based on a clear flow of inputs, backend operations, and system outputs. The step-by-step process is described below:

5.1. Inputs

1. **User Credentials:** The staff member or manager inputs registration details to create or log into an account.
2. **Product Search Queries:** The user enters a product name, category, or SKU to find specific items in the inventory.
3. **Transaction Selection:** The user selects a specific product ID and enters the quantity for a stock-in or stock-out transaction.
4. **Stock Details:** The user inputs product attributes, pricing, and supplier information to update or confirm the catalog logs.

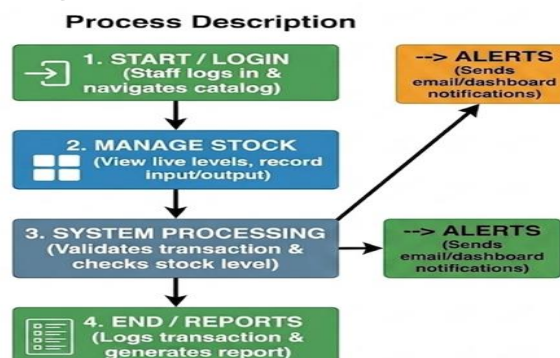
5.2. Operations

1. **Database Querying:** The backend searches the database for product items matching the searched SKU or category parameters.
2. **Real-Time Locking:** The system temporarily holds the transaction details and recalculates the updated stock quantity levels.
3. **Transaction Verification:** The server communicates with the internal logic engine to check if the transaction amount is successfully validated or logged.
4. **Status Management:** The system automatically switches the product alert state in the database between "Sufficient Stock", "Low Stock Alert", and "Out of Stock".

5.3. Outputs

- **Live Stock Display:** A color-coded, visual grid table showing live product inventory alert states (Green/Red/Yellow).
- **Transaction Summary:** A final stock-in or stock-out log receipt and confirmation slip generated for the business record.
- **Manager Dashboard:** An updated control panel showing real-time store capacity and detailed inventory logs for the warehouse owner.

System Flow Diagram



6. Hardware and Software Requirements

To develop and run the Inventory Management System smoothly, the following hardware configuration and software environments are required.

6.1. Hardware Requirements

- **Processor:** Intel Core i3 / AMD Ryzen 3 (or higher) to run the local server and compile code.
- **Memory (RAM):** Minimum **4GB** RAM (8GB recommended) to run the code editor, browser, and database together.
- **Storage:** At least **20GB** of available hard drive space for software dependencies and project files.
- **Internet Connectivity:** An active internet connection to load map APIs, download packages, and connect to cloud databases.

6.2. Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Code Editor:** **Visual Studio Code** for writing and debugging the application code.
- **Runtime Environment:** **Node.js** (LTS version) to run the backend server.
- **Database:** **MongoDB** (or MySQL) to store user profiles, product transaction logs, and livestock states.
- **Web Browser:** Google Chrome or Mozilla Firefox to view and test the front-end user dashboards.

7. Resources and Limitations

Resources

- **Free Tools:** Uses open-source software frameworks and standard web technologies (HTML, CSS, JavaScript, Node.js) that cost zero money to use.
- **Online Guides:** Access to free public documentation and forums makes solving code bugs very easy.
- **Basic Hardware:** Only requires a standard, everyday laptop and a normal internet connection to develop.

Limitations.

- **Needs Internet:** The app completely stops working if there is no active internet connection.
- **Manual Entry:** Relies on store staff to manually update their product counts unless expensive physical barcode scanners or automated RFID trackers are bought.

8.Future Work

- **Smart Sensors:** Install physical IoT sensors or RFID tags on warehouse shelves to auto-detect inventory levels without any manual inputs.
- **Expanded File Support:** Expand the verification and documentation system to support non-image file types, such as tracking PDFs, certificates, and media videos.
- **Mobile App:** Build a dedicated Android and iOS mobile app featuring quick item barcode scanning right from the warehouse floor.
- **Smart Forecasting:** Add basic demand forecasting algorithms to automatically predict future stock needs based on weekly sales trends.

9.Project Schedule (Gantt Chart)

PROJECT SCHEDULE (GANTT CHART)

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
Requirement Analysis						
Planning						
UI Design (HTML/CSS)						
JavaScript (Frontend Logic)						
Backend Development (Server-side)						
Database Design						
Testing						
Final Documentation						

10. Conclusion

The Inventory Management System provides an efficient, digital solution to the growing challenges of retail operations and chaotic stock management. By replacing outdated, manual logbooks with an automated web platform, the system allows staff to view live, color-coded product availability and secure inventory quantities instantly via real-time transaction tracking. Built entirely on a streamlined, single-language JavaScript stack using standard web front-end technologies and Node.js, the application ensures high performance, minimal server lag, and seamless data processing. Ultimately, this project reduces the time and resources wasted by businesses searching for product details, minimizes stockout errors, and equips warehouse operators with a hassle-free, automated dashboard to track counts and logs effortlessly.

11. References / Bibliography

- [1] W3Schools, “W3Schools Online Web Tutorials”. [Online]. Available: <https://www.w3schools.com/>. [Accessed: Jun. 19, 2026].
- [2] Mozilla Developer Network (MDN), “MDN Web Docs.” [Online]. Available: <https://developer.mozilla.org/>. [Accessed: Jun. 19, 2026].
- [3] MongoDB, Inc., “MongoDB Documentation.” MongoDB Docs. [Online]. Available: <https://www.mongodb.com/docs/>. [Accessed: Jun. 19, 2026].
- [4] Node.js Contributors, “Node.js Documentation.” [Online]. Available: <https://nodejs.org/en/learn>. [Accessed: Jun. 19, 2026].
- [5] E. Freeman and E. Robson, *Head First JavaScript Programming*. Sebastopol, CA, USA: O’Reilly Media, 2014.
- [6] A. Young, B. Meck, and M. Cantelon, *Node.js in Action*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2017.